

SECURITY REVIEW

FOR MYSTIC FINANCE



mystic[★]

INSIDE THIS REPORT

CONTENTS

01	AUDIT SUMMARY	03
02	PROTOCOL OVERVIEW	04
03	SCOPE	05
04	METHODOLOGY	06
05	RISK CLASSIFICATION	07
06	FINDINGS SUMMARY	08
07	DETAILED FINDINGS	09
	L-01 Incorrect Constructor Arguments Used in ZeroExOptimized Verification	09
08	DISCLAIMER	10
09	ABOUT KANN AUDITS	11

AUDIT SUMMARY

Mystic Finance engaged Kann Audits to perform a security review of the Mystic Finance smart contracts. From 12 – 13 February 2026, a team of security researchers performed a manual review of the in-scope contracts. All findings identified during the assessment are documented in this report.

TOTAL FINDINGS

1

ISSUES

CRITICAL	-
HIGH	-
MEDIUM	-
LOW	01
INFORMATIONAL	-

ENGAGEMENT DETAILS

PREPARED FOR	Mystic Finance
PREPARED BY	Kann Audits
DATE REPORTED	13 February 2026
REVIEW PERIOD	12 – 13 February 2026

PROTOCOL OVERVIEW

Mystic Finance is a DeFi lending and borrowing protocol that allows users to supply assets, borrow against collateral, and leverage positions, with support for liquid staking and tokenized real-world assets.

03 - WHAT WE REVIEWED

SCOPE

AUDITED COMMIT	139a4505e23028e7d6875fdbca294487eae4fdd4
----------------	--

#	FILE
01	deployment/scripts/deploy-optimized.js

METHODOLOGY

An engagement with Kann Audits involves a team of security researchers performing independent reviews of the codebase at the start of the engagement, followed by collaborative review sessions where auditors share findings and explore additional attack vectors to improve overall coverage.

The review included a line-by-line manual inspection of the in-scope codebase, along with analysis of the protocol architecture, access control mechanisms and state transitions. The team also assessed the code against common smart contract security best practices and industry standards.

STEP BY STEP

OUR SECURITY PROCESS

01

Kickoff & Context

A kickoff call to understand architecture, trust boundaries, threat assumptions and intended behaviour.

02

Scope & Preparation

Confirm the commit, files in scope, dependencies and the riskiest areas. We open a private channel between your team and our auditors.

03

AI First Scan

Kann AI, our proprietary model built to find security bugs, scans the codebase before the manual review starts.

04

Expert Security Review

Line-by-line analysis of exploit paths, invariant breaks, edge cases, accounting and integration risks.

05

Findings & Proofs

Each issue is confirmed, rated and backed by a proof of concept or a precise reproduction path.

06

Mitigation Review

We review every fix, confirm it resolves the root cause and flag partial or incorrect mitigations.

COVERAGE

THE REVIEW SPECIFICALLY CONSIDERED

- ✓ Testing against common and uncommon attack vectors
- ✓ Contract logic matching the client's intended behaviour
- ✓ Manual line-by-line review by experienced auditors
- ✓ Architecture and design analysis for secure component interactions and clear trust boundaries
- ✓ Compliance with industry best practices and known smart contract security standards
- ✓ Cross-referencing patterns with industry-standard implementations
- ✓ Access control verification
- ✓ State transitions and logic, to detect inconsistencies, unexpected behaviour or exploitation paths

RISK CLASSIFICATION

Severity combines the **impact** of an issue if exploited with the **likelihood** that the conditions for exploitation are met.

● CRITICAL	Directly exploitable with few or no preconditions, leading to theft or permanent loss of funds or loss of control of the system. Must be fixed before deployment.
● HIGH	Directly exploitable vulnerabilities that can lead to significant material loss or critical impact on the system and require immediate fixing.
● MEDIUM	Moderate loss of funds or impact on multiple users, but only under specific conditions. Exploitable without privileged access, but typically requires certain assumptions or edge cases.
● LOW	Issues with limited impact that do not put funds directly at risk, such as edge cases or deviations from documented behaviour.
● INFORMATIONAL	Non-exploitable findings such as best-practice deviations, cosmetic issues or documentation and compliance notes. They do not need urgent remediation but improve code quality and maintainability.

06 - ALL ISSUES

FINDINGS SUMMARY

● CRITICAL -	● HIGH -	● MEDIUM -	● LOW 01	● INFO -
TOTAL 01 FIXED 01 ACKNOWLEDGED 00				

ID	TITLE	SEVERITY	STATUS
L-01	Incorrect Constructor Arguments Used in ZeroExOptimized Verification	● LOW	● FIXED

DETAILED FINDINGS

L-01

Incorrect Constructor Arguments Used in ZeroExOptimized Verification

SEVERITY

● LOW

STATUS

● FIXED

■ DESCRIPTION

In the deployment script, the 'ZeroExOptimized' contract is deployed using the bootstrapper address obtained from 'fullMigration.getBootstrapper()'.

```
const zeroExOptimized = await deployContract(ZeroExOptimizedFactory, [await
  fullMigration.getBootstrapper()]);
console.log("ZeroExOptimized deployed to:", zeroExOptimized.address);
```

However, during the verification step, the script incorrectly uses the 'FullMigration' contract address as the constructor argument.

```
await verifyContract(hre, deploymentResults.contracts.ZeroExOptimized, [
  deploymentResults.contracts.FullMigration]);
```

This mismatch causes verification failures because the constructor arguments provided to the verification function do not match those used during deployment.

■ ROOT CAUSE

The verification logic does not account for the actual constructor parameter used when deploying 'ZeroExOptimized'. Instead of using the bootstrapper address returned by 'fullMigration.getBootstrapper()', it mistakenly uses 'fullMigration.address', resulting in incorrect verification data.

■ RECOMMENDATION

Pass bootstrapper as the constructor argument during verification.

```
const bootstrapper = await fullMigration.getBootstrapper();
deploymentResults.bootstrapper = bootstrapper;
await verifyContract(hre, deploymentResults.contracts.ZeroExOptimized, [
  deploymentResults.bootstrapper])
```

■ REMEDIATION

● FIXED

DISCLAIMER

A security audit can never guarantee the complete absence of vulnerabilities. Audits are a time, resource and expertise-bound effort in which we aim to identify as many issues as possible.

This review covers only the files listed in the Scope section at the audited commit. Code changed after that commit, external dependencies, deployment configuration and operational practices may introduce additional risk.

ABOUT KANN AUDITS

Kann Audits is a Web3 security audit company helping DeFi protocols, blockchain networks, and infrastructure teams build securely at every stage, from development to launch and beyond.



EXPERT SECURITY AUDITS

Human-led smart contract security audits built around the architecture, business logic, integrations and risk profile of complex Web3 protocols.



AI SECURITY SCAN

Kann AI extracts Solidity call graphs and returns function-level security signals for expert validation.



FORMAL VERIFICATION

Define security properties for critical protocol behaviour and evaluate whether the implementation satisfies those specifications.



INCIDENT RESPONSE

A direct triage path for suspected exploits, critical vulnerabilities, suspicious transactions and urgent remediation review.



KANN AUDITS
WEB3 SECURITY AUDITS

GET IN TOUCH

WEBSITE	kannaudits.com
PORTFOLIO	github.com/KannAudits
EMAIL	contact@kannaudits.com
X	@KannAudits
TELEGRAM	@KannAudits